CSE 351: Artificial Intelligence – Assignment 02

mb09640, ha10609

Fall 2026

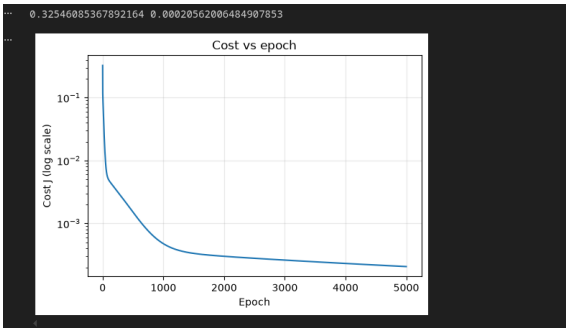
Question 1: Neural Networks as Universal Approximators

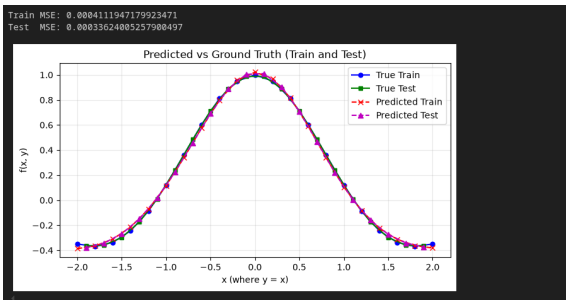
Baseline Model Results

A Multi-Layer Feedforward Neural Network (MFNN) was trained using backpropagation for 5000 epochs with 1 hidden layer (40 neurons), `tansig` activation, learning rate of 0.05, and initial bias of 0.5 to approximate $f(x, y) = e^{-\frac{x^2+y^2}{8}} \cos(\sqrt{x^2 + y^2} + c)$ with $c = 1 \times 10^{-3}$.

Metric	Value
Initial Cost J (Epoch 0)	0.325461
Final Cost J (Epoch 5000)	0.000206
Training MSE	0.000411
Testing MSE	0.000336

Table 1: MFNN Training and Testing Results





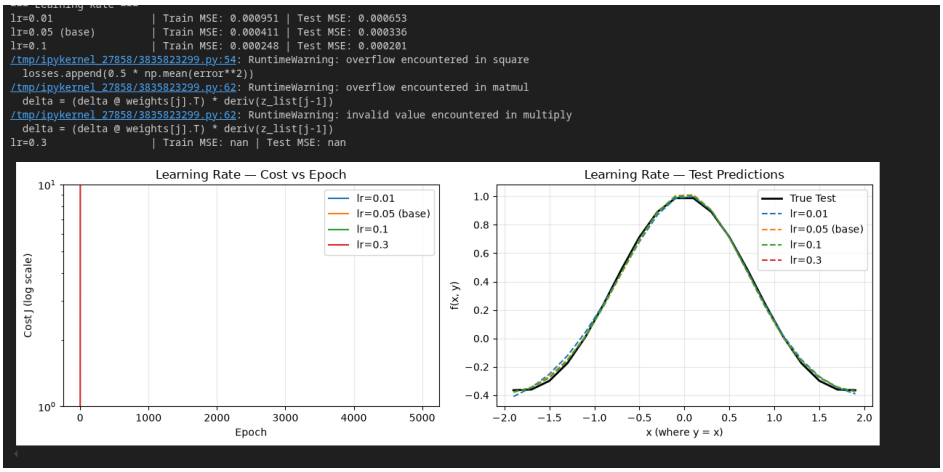
Fine-Tuning and Hyperparameter Findings

Comments:

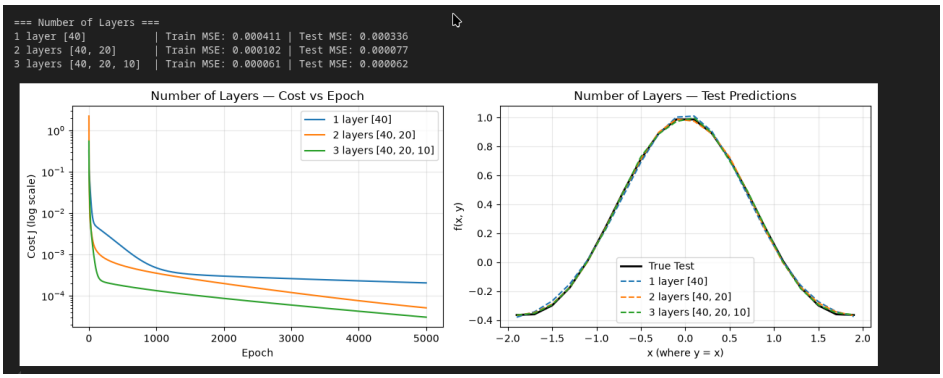
- **Learning Rates:** A learning rate of 0.01 converges more slowly (Test MSE: 0.000653). Increasing the rate to 0.1 improves convergence, achieving the lowest successful Test MSE of 0.000201. However, setting the learning rate too high to 0.3 causes numerical instability and overflow, resulting in `nan` values (this might be expected behavior and not a bug in my code as the model otherwise seems to run well).

Hyperparameter	Configuration	Train MSE	Test MSE
Learning Rate	lr = 0.01	0.000951	0.000653
	lr = 0.05 (Baseline)	0.000411	0.000336
	lr = 0.1	0.000248	0.000201
	lr = 0.3	nan	nan
Number of Layers	1 Layer: [40]	0.000411	0.000336
	2 Layers: [40, 20]	0.000102	0.000077
	3 Layers: [40, 20, 10]	0.000061	0.000062
Activation Function	tansig	0.000411	0.000336
	logsig	0.001950	0.001266
	relu	0.000715	0.000552
Initial Bias	bias = 0.0	0.000981	0.000730
	bias = 0.5 (base)	0.000411	0.000336
	bias = 1.0	0.000055	0.000043
	bias = 2.0	0.000543	0.000518

Table 2: Hyperparameters on Train and Test MSE

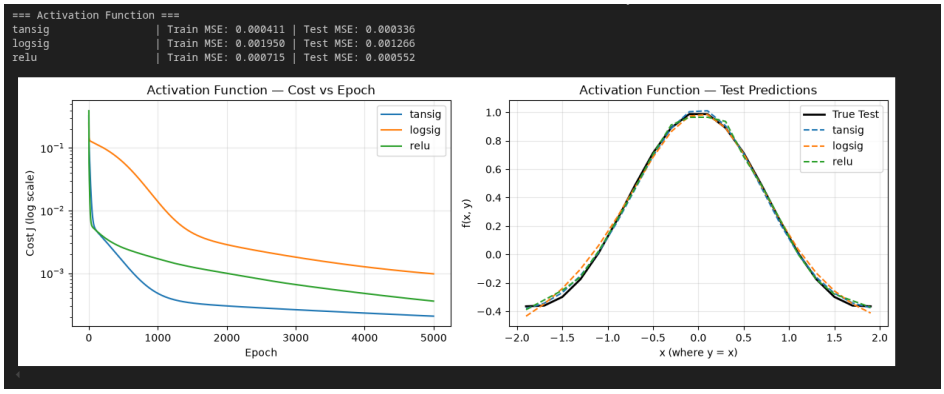


- **Number of Layers:** A single hidden layer of 40 neurons achieves a Test MSE of 0.000336. Adding deeper layers ([40, 20] and [40, 20, 10]) increases model capacity, further reducing Test MSE down to 0.000077 and 0.000062 respectively.

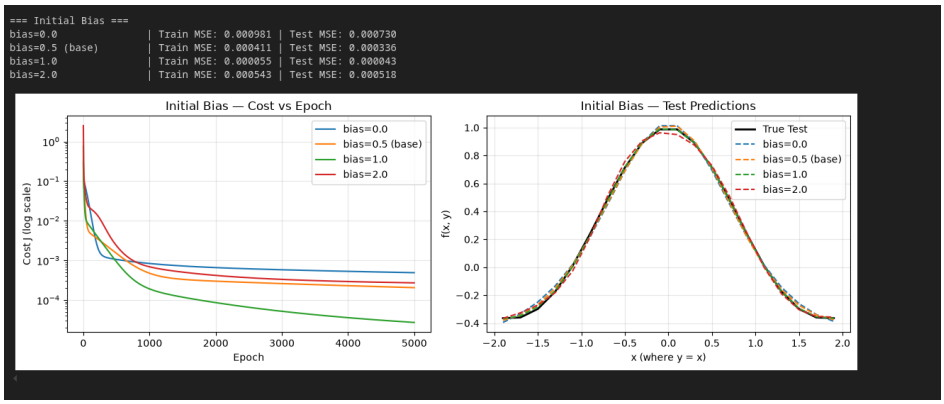


- **Activation Function:** `tansig` outperforms alternative activations, achieving the best gen-

eralization with a Test MSE of 0.000336. `logsig` exhibits higher error (Test MSE: 0.001266) due to slower convergence, while `relu` yields a Test MSE of 0.000552.



- **Biases:** Initializing with bias = 0.0 results in a higher Test MSE of 0.000730. The baseline bias of 0.5 achieves 0.000336, while a bias of 1.0 yields the optimal Test MSE of 0.000043. Pushing the initial bias higher to 2.0 degrades performance to 0.000518.



Question 2: Decision Tree Induction

For this part we wrote our own decision tree in Python. At every node we compute the weighted Gini for each leftover attribute and split on the one with the smallest value. Smaller weighted Gini means the groups after the split are cleaner. The three datasets are in `DATASET.mat`.

a) GINI Index

Dataset	a_1	a_2	a_3	a_4	a_5	a_6
Data1	**0.1863**	0.4935	0.4949	0.4620	0.4861	0.4664
Data2	0.4928	0.4899	0.4885	**0.4834**	0.4900	0.4940
Data3	0.4690	0.4639	0.4725	**0.4447**	0.4685	0.4691

Table 3: Weighted GINI on the training sets. Bold means best attribute for the root.

So the root should be a_1 for Data1, and a_4 for both Data2 and Data3.

Data1 is the easy one. a_1 jumps out right away (0.1863). Everything else is still around 0.46 to 0.49. That fits what the prompt says about Data1 being based on simple equality and OR conditions.

Data2 looks basically flat. All six numbers are near 0.49, and even the winner (a_4 at 0.4834) is barely better than the others. Since Data2 is parity based, the label depends on a pattern across attributes, not on one column by itself. Gini only checks one attribute at a time, so it does not have much to work with here.

Data3 is in between. a_4 wins again (0.4447), so Gini still finds something useful, but the gap is smaller than on Data1. The prompt says Data3 has harder rules and 5% noise. In this kind of synthetic data that usually means about 5% of the class labels are flipped on purpose, so a few rows are marked with the wrong class. The attributes stay the same. That makes the training data a bit messy and can hurt how clean the Gini scores look.

Overall, Gini is fine when one attribute actually carries the answer (Data1). It is pretty weak on Data2. On Data3 it still picks a reasonable root, but the noise and harder rules make the scores less separated.

b) Tree Construction (Data1)

We only had to build the full tree for Data1. Same Gini rule as above: keep splitting until a branch is pure, or until every remaining attribute is constant. Then we ran the tree on the Data1 test set and plotted it with NetworkX.

Metric	**Value**
Root attribute	a_1
Data1 test accuracy	1.0000 (100%)

Table 4: Data1 tree results

Test accuracy was 100

Question 3: Image Classification for Object Recognition

We used the CIFAR-10 files in `cifar-10-batches-py`. Each image is 32×32 with 3 color channels, so after flattening we get 3072 features. We divided pixels by 255 and then used `StandardScaler` (fit on train only).

We did not train on all 50,000 images for the main table. Linear SVM and kNN were taking too long on the full set, so we randomly picked 10,000 training images with seed 42. Testing still used all 10,000 test images.

Settings we used:

- Decision Tree: `max_depth=15`
- ANN: one hidden layer with 80 neurons, `max_iter=30`, early stopping on
- SVM: linear kernel
- kNN: $k = 3$

ANN wins by a decent margin (about 41

Classifier	Accuracy	Precision	Recall	F-measure
(a) Decision Tree	0.2445	0.2438	0.2445	0.2434
(b) Artificial Neural Network	0.4093	0.4094	0.4093	0.4090
(c) Support Vector Machine (SVM)	0.3026	0.3086	0.3026	0.3032
(d) K-Nearest Neighbour (kNN)	0.2850	0.3820	0.2850	0.2709

Table 5: CIFAR-10 scores with 10,000 training images and the full test set. Precision, recall, and F-measure are macro averages.

We also tried other hyperparameters on the same 10k split (tree depth 20, ANN with 100 units and 20 iterations, $k = 5$). Accuracies were 0.2472, 0.4173, 0.3026, and 0.2934. A bit better in places, but nothing dramatic. Changing these settings a little does not fix the raw-pixel problem.

Before we switched to 10k, we did get the tree and ANN to finish on the full 50k training set. Those accuracies were 0.2755 and 0.4985. So more data helps, especially for the ANN. We stopped the full run because the linear SVM was going to take too long, which is why the comparison table above uses the 10k subsample.